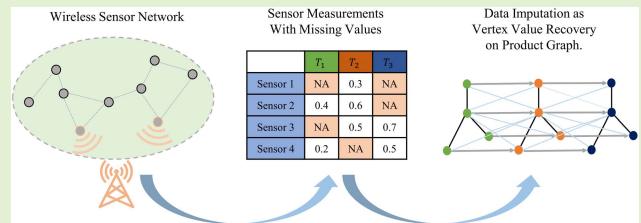


A Graph-Based Approach for Missing Sensor Data Imputation

Xiao Jiang^{ID}, Zean Tian^{ID}, and Kenli Li^{ID}, *Senior Member, IEEE*

Abstract—The Internet of Things (IoT) oriented intelligent services require high-quality sensor data delivery in the wireless sensor networks (WSNs). However, either due to the sensor malfunctions and commutation errors or simply due to the expensive overhead for making full data forwarding, data corruption and loss is relatively common in WSNs, which adversely affects the data quality and the further decisions taking from data. Motivated by the emerging field of graph signal processing (GSP), we propose to impute the missing values in wireless sensor networks based on the topological information carried in the product graph. The proposed solution captures the joint time-space dependencies among the sensor data through a spatiotemporal (ST) graph, which is a time-vertex graph constructed by taking a strong product of a temporal graph and a spatial sensor network graph. Then, the sensory data are mapped onto the vertices of the ST graph and the spatial-temporal nature of sensor data can be further characterized by the notion of smoothness used in GSP. Moreover, instead of imputing with a given spatial graph, we propose a graph learning-based imputation framework to infer underlying space dependencies between the sensors and thus enhance the data imputation performances. Finally, we validate the proposed recovery method using real-world sensor network datasets. The results demonstrate the superior performance of our proposed graph-based method in sensor data imputation, especially when massive sensor data are lost.

Index Terms—Graph signal processing, missing data recovery, product graph, wireless sensor networks.



I. INTRODUCTION

WIRELESS sensor networks (WSNs) are regarded as a core and essential technology to enable the Internet of Things (IoT) paradigm [1]. Recently significant advancements in sensing and wireless communications have facilitated WSNs to be utilized in a plethora of promising IoT applications such as smart grid, intelligent traffic management, health care, environmental monitoring, etc [2], [3]. In WSNs, a group of sensor nodes is installed to track product movements or measure the local environmental conditions, and the networked sensors wirelessly forward the data to the sink (base station), which then transmits the collected data subsequent database through other networks (e.g. Internet). Further, the application servers store the data, conduct data analysis using various signal processing and machine learning algorithms, and finally provide analysis results to end-users [4].

Manuscript received July 9, 2021; revised August 14, 2021; accepted August 16, 2021. Date of publication August 23, 2021; date of current version October 18, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant 51661005. The associate editor coordinating the review of this article and approving it for publication was Prof. Pierluigi Salvo Rossi. (Corresponding author: Zean Tian.)

The authors are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China (e-mail: tianzean@hnu.edu.cn).

Digital Object Identifier 10.1109/JSEN.2021.3106656

Effective and reliable data delivery is a prerequisite for WSNs to provide intelligent services, however, data loss is inevitable in WSNs [5]. In practical applications, most sensor nodes are typically low-cost and energy-limited and placed in a harsh and unattended environment. It is almost impossible to fix the breakdown rapidly when the sensor node is malfunctioning or the battery is depleted, which typically renders the readings intermittent. Apart from this, because of the interference and malicious attack of the communication environment, packet loss in the process of data transmission will also result in a large number of data losses. On the other hand, in large monitoring WSNs, a tremendous amount of sensor readings is generated and delivered to the data collector, resulting in a lot of energy spent by receiving and forwarding data. To save energy and maximize the lifetime of a sensor network, sampling algorithms such as adaptive sampling [6] and compressed sensing [7] are usually used to actively reduce the data transmission interval of sensor nodes and the number of sensor nodes. However, the presence of missing values in collected sensor data, whether caused by non-human factors or human factors, will severely degrade the performance of the associated task further causing big problems in the decision-making process. Therefore, enhancing the integrity and accuracy of sensor data is an important step before performing further data analysis.

An early used sensor data imputation approach is the mean method that interpolates the missing values with the

average value of the observed values. Moreover, K-Nearest Neighbors (KNN) algorithm has long been used to recover missing sensor data [8]. The value of a given sensor is replaced by weighting and averaging the values of its closest k neighbors. As the advantages of simplicity and ease of implementation, the mean method and KNN usually serves as the baseline. Besides, the probabilistic method such as expectation-maximization (EM) was adopted to complete the missing data by alternately estimating the conditional expectation in E-step and the model parameters in M-step [9]. However, these approaches do not consider the spatial and temporal relationships among sensory data and the recovery quality is generally poor thus limiting its application in practical scenarios. To overcome this limitation, Gruenwald *et al.* propose to recover the value at a given sensor by using the known values at the geospatially adjacent sensors [10]. To include richer spatial relations among sensors, spatiotemporal multi-view learning [11] has been developed to recover the missing values based on the data on nearest spatial and temporal neighbors. Some researchers consider the spatial-temporal data imputation problem as a low-rank matrix completion (LRMC) task and estimate the missing values by finding a suitable low-rank approximation of the original incomplete data matrix [12], [13]. As an extension of LRMC, tensor composition have also been applied to deal with high dimensional data in sensor network [14], [15]. Moreover, reference [16] proposed to recover data by firstly clustering the sensors nodes and then using matrix factorization (MF) to learn latent factors. Nevertheless, both of them only catch the surface characteristics, falling short in inferring the inherent dependencies of the spatiotemporal sensor data.

Recently, graph signal processing (GSP) provides a novelty tool to process sensor network data collected at the spatially irregular domain [17]. In the graph representation, each vertex models a sensor node in the environment, and the data sequence on each node forms a vertex-indexed signal referred to as the graph signal. Given the network topology associated with a sensor network, data measurements that are similar to adjacent vertices contribute to a smooth (low-pass) graph signal, and then the high-pass filter can be utilized to detect the abnormal values [18], [19]. Furthermore, such a smoothness model allows for efficiently reconstructing graph signals from observations sampled from a subset of vertices [20], [21], which could lead to considerable energy and bandwidth reductions in WSNs [22]. Lately, reference [23] developed the Graph Neural Network strategy to handling the missing sensor problem in topology-aware IoT applications, where the neural message passing mechanism is used to transmit the information flow from available sensors to missing sensors during reconstruction. However, they assume that the sensor graph is prior known, which may not always be the case. In certain situations, the phenomena that explain the proximity of sensors are latent and the underlying graph needs to be inferred from data [24]. Jablonski *et al.* [25] firstly introduced the graph learning method developed in [26] to infer a sensor network graph from the ozone dataset. The hidden behavior and system patterns can be identify based on the learned graph. Recently, Ferrer-Cid *et al.* compare the applications

of the statistic-based and smoothness-based graph learning techniques using the air pollution data [27]. The results show that the graph learned from the signal smoothness-based model behaves better in signal reconstruction.

In addition to WSNs, some related works in other applications such as recommendation systems also leverage the graph structure to handle the data completion problems [28]–[30]. For example, Kalofolias *et al.* [28] use two independent auxiliary graphs to represent the existing relationships between rows and columns of an incomplete matrix, and proposed a graph regularization based matrix completion model wherein the smoothness terms are imposed over the row graph and column graph. Then, the authors in [29] further employed the graph convolution neural network to learn the representation of row and column through node information aggregation in these two individual graphs, and to impute the incomplete matrix by MF method. However, these methods also assume the prior availability of graphs, thus they will suffer from performance degradation in practice, while the incorporation of graph learning can alleviate this issue [31].

Nevertheless, though the graph-based techniques have been successful for many tasks, so far the potential of completing the missing sensor data with complex spatiotemporal correlations is yet unexplored. To fill this gap, we develop a graph-based algorithm for imputing the spatiotemporal data in the wireless sensor network. The main contributions of this paper are as follows:

- We propose a time-vertex model that incorporates the novel GSP tools to represent the spatiotemporal sensor data. Such data are indexed by the vertices of the spatiotemporal graph and the underlying sophisticated functional dependencies of data are encoded as the edge weights. To better capture the inherent spatiotemporal structure and save the computational loads, we factorize the spatiotemporal graph of large dimensions as the product of two smaller factor graphs, i.e., spatial graph and time-series graph.
- Combing the prior assumption of spatiotemporal smoothness in the local sense and the low-rank property in the global sense, we formulate a joint graph signal recovery and low-rank matrix completion problem to handle the sensor data imputation.
- Instead of performing the imputation with a pre-defined spatial graph, we propose to infer the underlying spatial correlations that best fit the sensor data. The corresponding edge weights of the spatial graph are simultaneously learned from data while considering temporal structure.
- Experiments on the real-world dataset are conducted to evaluate the performance of the proposed imputation algorithm and compare it with state-of-art data recovery approaches. The experiment results demonstrate the superiority of our developed method.

The remainder of this paper is structured as follows. Section II introduces the overall data imputation framework and formulates the objective for completing the spatiotemporal sensor data. The solver of this optimization problem is presented in section III. Section IV shows the numerical results. Section V concludes this paper.

Notation: Scalars are denoted by standard lowercase or uppercase letters, column vectors are represented by bold lowercase letters, and matrices are denoted by bold uppercase letters. $\odot$ denotes the Hadamard product operator. The Cartesian product, Kronecker product and the Strong product of matrices $\mathbf{A}$ and $\mathbf{B}$ are denoted by $\mathbf{A} \oplus \mathbf{B}$, $\mathbf{A} \otimes \mathbf{B}$ and $\mathbf{A} \boxtimes \mathbf{B} = \mathbf{A} \otimes \mathbf{B} + \mathbf{A} \oplus \mathbf{B}$, respectively. $\mathbf{A}^T$, $\mathbf{A}^{-1}$, $\text{tr}(\mathbf{A})$ denote the transpose, inverse, and trace of matrix $\mathbf{A}$, respectively. $\|\mathbf{A}\|_F$ is the Frobenius norm of $\mathbf{A}$. $\|\mathbf{A}\|_{1,\text{off}}$ denotes the ℓ_1 -norm of the off-diagonal entries of matrix $\mathbf{A}$. $\mathbf{1}_n$ is the n -dimensional all-ones vector.

II. JOINT TIME-VERTEX FRAMEWORK FOR MISSING DATA RECOVERY

In this section, we first present the overall framework of the proposed GSP-based sensor data imputation method. Then we describe the graph construction technique for modeling spatiotemporal correlations of sensor data and finally formulate joint graph learning and the data imputation problem.

A. Problem Formulation

Suppose that we have N sensor nodes distributed in the environment and the sensor data is collected at T successive regular intervals of unit length, then the t^{th} measurements are given by the N -dimensional vector $\mathbf{x}_t \in \mathbb{R}^N$. The overall sensor data can be organized as the matrix $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T] \in \mathbb{R}^{N \times T}$, where the rows correspond to the time series of each sensor, and columns correspond to data from all sensors per time slot. Furthermore, we can rearrange the data matrix $\mathbf{X}$ into the vector $\mathbf{x} = \text{vec}(\mathbf{X}^T) \in \mathbb{R}^{NT}$ such that $X_{n,t} = x_{t+(n-1)T}$. It should be noted that throughout the paper, we refer to both $\mathbf{X}$ and its vectorized version $\mathbf{x}$ as spatiotemporal sensor data.

As mentioned before, due to the sensor failure or transmission interruption, collected sensor data will contain missing values, that is, only a subset of $\mathbf{X}$ are valid sensing data. If the collected sensor data is denoted by matrix $\mathbf{Z} \in \mathbb{R}^{N \times T}$, the limited observation model is given as follows:

$$\mathbf{Z} = \mathbf{M} \odot (\mathbf{X} + \mathbf{E}). \quad (1)$$

Here, the measurement imperfections are assumed to be the Gaussian white noise and summarized in the error term $\mathbf{E} \in \mathbb{R}^{N \times T}$. The matrix $\mathbf{M} \in \mathbb{R}^{N \times T}$ is a missing indicator for the observation matrix $\mathbf{Z}$, where each entry $m_{n,t}$ is defined as the below binary mask:

$$m_{n,t} = \begin{cases} 1, & \text{if } X_{n,t} \text{ is a known value,} \\ 0, & \text{if } X_{n,t} \text{ is a missing value,} \end{cases} \quad (2)$$

where $X_{n,t}$ is the actual data of a sensor n at time t .

B. Graph Signal Recovery

The analysis of spatiotemporal sensor data is a difficult problem since significant spatiotemporal patterns exist and are hard to be captured. However, a networked structure allows for explicit modeling such sophisticated relations underlying the data in the form of pairwise relationships [25].

Formally, let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a weighted and undirected graph, where $\mathcal{V}$ is the set of vertices with cardinality $|\mathcal{V}|$, and $\mathcal{E}$ is the edge set that reveals the connections between the nodes. To encode the complex spatiotemporal correlations occurring in sensor data, we construct a spatiotemporal graph by firstly mapping both the regular time points and the spatially irregular sensor locations to the graph vertices. There is a one-to-one function $\phi : \mathcal{V} \rightarrow [N] \times [T]$ that maps each node of a spatiotemporal graph to a pair of time-space index. Hence, we have $|\mathcal{V}| = NT$. Then, a spatiotemporal sensor data $\mathbf{x}$ that consists of a collection of NT real values is a static graph signal residing on the graph $\mathcal{G}$, in which the signal values x_i is associated to the i -th node of the graph. Additionally, the correlations across space and time can be described by the connection of the spatiotemporal graph and further represented by the weighted adjacency matrix $\mathbf{W} \in \mathbb{R}^{NT \times NT}$ whose element denotes the weight of the edge between adjacent vertices. Here, $W_{i,j} = 0$ implies no connection and $W_{i,j} > 0$ between vertex i and vertex j refers to their similarity or dependency. Since the graph is assumed to be undirected, we have $W_{i,j} = W_{j,i}$. Based on the adjacency matrix, we can also define an alternative graph representation known as combinatorial graph Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{W}$, where $\mathbf{D} = \text{diag}(\mathbf{W}\mathbf{1}_{NT})$ is the graph degree matrix. Thus, a valid Laplacian matrix satisfies: $L_{i,j} = L_{j,i} \leq 0, \forall i \neq j$ and $\sum_{j=1}^N L_{i,j} = 0, \forall i$. We denote $\mathbf{L} \in \mathcal{L}$ if these conditions are satisfied.

As we encode the spatiotemporal correlations of sensor data into a spatiotemporal graph, the data imputation task can be naturally formulated as the graph signal recovery problem. Our goal now is to reconstruct the complete static graph signal from the samples on the subset of vertices. A standard solution to deal with this problem generally contains the following two-step process:

1) Step 1: Spatiotemporal (ST) Graph Learning: Firstly, we would like to estimate an optimal weighted and undirected ST graph $\mathcal{G}$ such that the weights of all possible edges can precisely quantify the underlying complex spatial-temporal correlations. The more related data points must have strong connections in the graph $\mathcal{G}$. Popular methods in GSP typically assume that the graph data is smooth with respect to the unknown graph in the sense that the connected vertices have similar values [24], [26]. The smoothness of graph signals over one graph can be quantified as:

$$S(\mathbf{x}) = \mathbf{x}^T \mathbf{L} \mathbf{x} = \frac{1}{2} \sum_{i,j} W_{i,j} (x_i - x_j)^2, \quad (3)$$

which is a summation of the squared Euclidean (based on ℓ_2 norm) distance between graph signals in neighborhood nodes. Considering that the spatial-temporal sensor data requires the value $S(\mathbf{x})$ is small, the underlying ST graph learning thus can be formulated as the search of a valid graph Laplacian matrix $\mathbf{L}$ which minimizes the following Laplacian quadratic form:

$$\min_{\mathbf{L} \in \mathcal{L}} \mathbf{x}^T \mathbf{L} \mathbf{x} + f(\mathbf{L}), \quad (4)$$

where $f(\mathbf{L})$ is a regularization function to avoid trivial solutions (e.g., zero matrix), or to impose further structure depending on the prior information of the graph. For example, real-world connections are usually more local, and thus not all graph nodes in real-world databases are related to each other. To enforce the local connectivity in the graph, the ℓ_1 norm regularizer $f(\mathbf{L}) = \|\mathbf{L}\|_{1,\text{off}}$ that acts on the off-diagonal entries of the Laplacian matrix can be adapted to learn a sparse graph. The graph topology behind the spatiotemporal sensor data $\mathbf{X}$ can be learned directly by solving (4).

2) Step II: Graph Signal Recovery: Given the discovered graph topology, the next step is to impute the missing sensor data based on a certain metric in the graph domain. Since the Laplacian matrix of the discovered spatiotemporal graph is symmetric and positive semidefinite, its eigendecomposition admits $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$, where $\mathbf{U}$ is a matrix having an orthonormal set of eigenvectors $\{\mathbf{u}_i\}_{i=1}^{NT}$ as columns and $\mathbf{\Lambda}$ is the diagonal matrix of eigenvalues. Therefore, the Graph Fourier Transform (GFT) of the sensor data $\mathbf{x}$ is defined as the projection onto the subspace spanned by the eigenvectors of the graph Laplacian matrix, i.e., $\mathbf{h} = \mathbf{U}^T\mathbf{x}$, where $\mathbf{h}$ is the GFT coefficients [32]. In this sense, the eigenvectors $\mathbf{U}$ are interpreted as graph Fourier basis and the diagonal elements of $\mathbf{\Lambda}$ as frequencies. Then, the spatiotemporal sensor data $\mathbf{x}$ is K -bandlimited graph signal if the number of non-zero elements in its frequency coefficients vector $\mathbf{h}$ is less or equal to $K < NT$. This band-limited property enables us to impute the missing sensor data from the incomplete observations when the number of samples is not smaller than the bandwidth K . However, the band size K may not be known in practice and data imputation approaches are not straightforward from finding K .

More generally, instead of directly enforcing bandlimited structure in the spectral domain, one can formulate an optimization problem for graph signal reconstruction by regularizing the smoothness of the graph signal as the penalty term. That is, we can impute the sensor data by solving the following unconstrained problem:

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{M} \odot (\mathbf{X} - \mathbf{Z})\|_F^2 + \frac{\alpha}{2} S(\mathbf{x}), \quad (5)$$

where α tunes the relative importance of the smoothness term defined in (3). When α is larger, more emphasis is put on the data smoothness over the minimization process, otherwise, more attention will be paid to the data fitting. Note that (5) is a convex quadratic problem, and it admits a closed-form solution. Hence, we get the reconstructed sensor data:

$$\mathbf{x} = (\text{diag}(\text{vec}(\mathbf{M})) + \alpha \mathbf{L})^{-1} \text{vec}(\mathbf{Z}). \quad (6)$$

In (6), the matrix $\text{diag}(\text{vec}(\mathbf{M})) + \alpha \mathbf{L}$ can be interpreted as a graph filter, where a larger α will result in a smoother signal.

Since the aforementioned strategy is not designed specifically for spatial-temporal sensor data imputation problem, if we apply it directly without exploring the underlying properties of data, there will have following drawbacks. Firstly, its graph learning step takes as the input the complete set of data observed at all vertices of the spatiotemporal graph,

which may fall short if the available data in the sensor network contains massive missing values. Furthermore, prior information about the spatiotemporal dependencies, like the correlation between two consecutive time samples, is not taken into account when learning the graph structure. Then, it will output a feature-based similarity graph, which destroys the inherent structure of the spatiotemporal graph and may lead to an inaccurate or even erroneous graph. Eventually, the performance of subsequent missing data recovery will be deteriorated severely when we interpolate with the learned graph. Secondly, it suffers from the high computational load when dealing with long-time sequence data over a large-scale sensor node. Reconstructing the data involves computing the inversion of a matrix of size $NT \times NT$, which incurs computational complexity $\mathcal{O}(N^3T^3)$, and the state-of-art graph learning algorithms will cost $\mathcal{O}(N^2T^2)$ for learning spatiotemporal graph, hence the storage and computational complexity is prohibitive for a real-world sensor dataset. To tackle these issues, we next adopt the concept of product graph to propose our missing data imputation method that not only leverages the structure inherent in product graphs for improved performance but also provides savings in memory and computational complexity.

C. Proposed Approach

The spatiotemporal sensor data collected from different locations and times are inherently multidomain (i.e., spatial dimension and temporal dimension). To further exploit the structure underlying such multidomain data, we can assume that the spatiotemporal graph is composed of the product of two or more small graphs. Recently, in order to capture the local interactions in data, the work in [33] focus on learning Cartesian graph from multidomain data. This offers a flexible and useful way to factorize the graph of larger dimensions such that each factor graph corresponds to one domain. Hence, we can firstly construct a spatial sensor network graph $\mathcal{G}_1 = (\mathcal{V}_1, \mathcal{E}_1)$ with vertex set of cardinality $|\mathcal{V}_1| = N$ to model the spatial correlations between the sensor nodes, and a time-series graph $\mathcal{G}_2 = (\mathcal{V}_2, \mathcal{E}_2)$ with vertex set of cardinality $|\mathcal{V}_2| = T$ to model the temporal dependencies. The vertices of graph $\mathcal{G}_1$ and $\mathcal{G}_2$ represent different sensors and time steps. Formally, the spatiotemporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ can then be factorized as the product (denoted by the symbol $\diamond$) of $\mathcal{G}_1$ and $\mathcal{G}_2$, i.e., $\mathcal{G} = \mathcal{G}_1 \diamond \mathcal{G}_2$, where vertex set of cardinality $|\mathcal{V}| = |\mathcal{V}_1||\mathcal{V}_2|$ and edge set $\mathcal{E} \subseteq \mathcal{V}_1 \times \mathcal{V}_2$ are defined based on the rules of graph product. We denote the weighted adjacency matrix, degree matrix and Laplacian matrix of graph $\mathcal{G}_k$ as $\mathbf{W}_k$, $\mathbf{D}_k$ and $\mathbf{L}_k = \mathbf{D}_k - \mathbf{W}_k$ for $k = 1, 2$, respectively.

A straightforward way to build the spatial sensor network graph $\mathcal{G}_1$ is based on the distance between pairs of sensors, since data of geographically nearby sensor nodes are usually close to each other. For instance, one can determine the edge weights of $\mathcal{G}_1$ through a thresholded Gaussian kernel weighting function, i.e., $\exp(-d(i, j)^2/2\sigma^2)$ where $d(i, j)$ is the distance between sensor nodes and σ is the kernel width parameter. On the other hand, to construct the temporal graph $\mathcal{G}_2$, we use a known line graph that connects two consecutive time steps

with the edge weights given by

$$w_2(i, j) = \begin{cases} c, & |i - j| = 1, \\ 0, & |i - j| \neq 1. \end{cases} \quad (7)$$

where the constant $c > 0$ determines the strength of the temporal correlation between two consecutive time samples. Then, the Laplacian matrix of time graph is given by the following tridiagonal matrix

$$\mathbf{L}_2 = c \begin{bmatrix} 1 & -1 & 0 & \cdots & 0 & 0 \\ -1 & 2 & -1 & \cdots & 0 & 0 \\ 0 & -1 & 2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & -1 & 2 \end{bmatrix}_{T \times T}. \quad (8)$$

There are three most commonly studied product graphs and their respective edges are connected in different modes [32]. We consider the following product graphs.

- 1) *Cartesian Product Graph* $\mathcal{G}^c = \mathcal{G}_1 \oplus \mathcal{G}_2$: The adjacency and graph Laplacian matrices are obtained as $\mathbf{W}^\oplus = \mathbf{W}_1 \oplus \mathbf{W}_2$ and $\mathbf{L}^\oplus = \mathbf{L}_1 \oplus \mathbf{L}_2$, respectively.
- 2) *Kronecker Product Graph* $\mathcal{G}^k = \mathcal{G}_1 \otimes \mathcal{G}_2$: The adjacency and graph Laplacian matrices are given by $\mathbf{W}^\otimes = \mathbf{W}_1 \otimes \mathbf{W}_2$ and $\mathbf{L}^\otimes = \mathbf{D}_1 \otimes \mathbf{D}_2 - \mathbf{W}_1 \otimes \mathbf{W}_2$, respectively.
- 3) *Strong Product Graph* $\mathcal{G}^s = \mathcal{G}_1 \boxtimes \mathcal{G}_2$: The adjacency and graph Laplacian matrices are given by $\mathbf{W}^\boxtimes = \mathbf{W}^\otimes + \mathbf{W}^\oplus$ and $\mathbf{L}^\boxtimes = \mathbf{L}^\otimes + \mathbf{L}^\oplus$, respectively.

An illustration of the product graph is shown in Fig. 1. As observed, the measurement $X_{n,t}$ of n^{th} sensor at t^{th} time step is correlated not only to the measurements $\{X_{n',t}\}_{n' \in \mathcal{N}_n}$ of neighbor sensors $\mathcal{N}_n = \{n' | W_1(n, n') \neq 0\}$ at current time step, but also to the measurements $X_{n,t-1}$ of the current sensor at previous time step and $X_{n,t+1}$ at next time step in the case of Cartesian product graph. For Kronecker product graph, $X_{n,t}$ correlates to the measurements $\{X_{n',t-1}\}_{n' \in \mathcal{N}_n}$ of neighbors sensor at previous time step and $\{X_{n',t+1}\}_{n' \in \mathcal{N}_n}$ at next time step. For the Strong product graph, it composes the graph by combining both the Cartesian product graph and the Kronecker product graph, and thus the edges are the union set of edges in these two graphs. It is noted that by definitions of the graph structure, the Kronecker and Strong product graphs have more edges than the Cartesian product graph, and thus may encode richer information of spatial-temporal correlations in sensor data. On the other hand, the Cartesian product graph enforces the connections in more stringent circumstances, which may contain less noise in defining these correlations, especially when the given factor graph contains some error edges.

After setting the product structure of the spatiotemporal graph, we can assume that the sensor data are smooth with respect to the constructed product graph and further exploit this property in the data imputation process. Additionally, the sensor data matrix also exhibits low-rank property. Thus, the problem boils down to recover the sensor data matrix that is smooth on the spatiotemporal product graph and simultaneously low rank. We define the following convex problem for

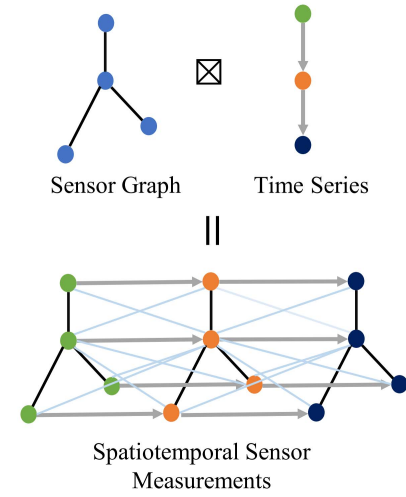


Fig. 1. The sensor measurements are indexed through a product graph. The Cartesian product graph includes the edges shown in black and green, and the Kronecker product includes the edges shown in black and wathet. The Strong product graph includes all edges.

missing sensor data imputation:

$$\min_{\mathbf{X}} \frac{1}{2} \|\mathbf{M} \odot (\mathbf{X} - \mathbf{Z})\|_2^2 + \frac{\alpha}{2} \mathbf{x}^T \mathbf{L}^\diamond \mathbf{x} + \mu \|\mathbf{X}\|_*, \quad (9)$$

where $\|\cdot\|_*$ denotes the nuclear norm and $\mu \geq 0$ controls the rank of the data matrix. Here, $\mathbf{L}^\diamond$ is the Laplacian matrix of one of the three types of product graph and can be selected according to the application scenario.

Generally, the geographical graph generated based on the distance between the sensor nodes may be defective for modeling the spatial similarity of measurements between those sensors. Some factors like the existence of geographical barrier might affect the values recorded in the sensor readings (e.g., in temperature measurements) [17], [26]. Therefore, instead of building the product graph with a given factor graph of the spatial domain, we turn to recover a sensor graph that reflects the latent dependencies between sensors in the spatial domain from the incomplete sensor data. That is, we aim to infer a graph in which the sensors that have similar evolution patterns are connected. By doing this, we can estimate both the missing data and underlying spatial graph at the same time and thus enhance the data imputation performance. In order to choose a good prior on the graph Laplacian for an efficient update of the sensor graph, we use $f(\mathbf{L}_1) = \text{tr}(\mathbf{1}^T \log(\text{diag}(\mathbf{L}_1))) + \|\mathbf{L}_1\|_{1,\text{off}}$ proposed in [34] to enforce a connected graph, while controlling graph sparsity. Formally, we propose the following joint data imputation and graph learning problem:

$$\min_{\mathbf{X}, \mathbf{L}_1 \in \mathcal{L}} \frac{1}{2} \|\mathbf{M} \odot (\mathbf{X} - \mathbf{Z})\|_2^2 + \frac{\alpha}{2} \mathbf{x}^T \mathbf{L}^\diamond \mathbf{x} + \mu \|\mathbf{X}\|_* - \beta_1 \text{tr}(\mathbf{1}^T \log(\text{diag}(\mathbf{L}_1))) + \beta_2 \|\mathbf{L}_1\|_{1,\text{off}}, \quad (10)$$

where $\beta_1 > 0$ and $\beta_2 > 0$ are regularization parameters. Increasing β_1 can enhance the connectivity between highly correlated sensor, while increasing β_2 makes the solution more sparse. The smoothness term in (10) is convex in $\mathbf{x}$ and linear in $\mathbf{L}^{\mathcal{G}_1 \diamond \mathcal{G}_2}$, however, it is non-convex on these two variables jointly. Thus, the objective function (10) is non-convex and

finding its global minimum is difficult. This motivates an alternation minimization algorithm to find the local minimum of (10) by iteratively optimizing with respect to $\mathbf{x}$ and $\mathbf{L}^{\mathcal{G}_1 \odot \mathcal{G}_2}$. In the following, we only show the algorithm on the Kronecker product graph and the derivations can easily be extended to the Cartesian and Strong product graph.

III. ALGORITHM

In an alternating manner, the joint sensor data imputation and graph learning solution iterate over two stages. The steps are summarized as follows:

A. Sensor Data Recovery

Given the Laplacian matrix obtained from the last iteration, the optimization problem for imputing missing sensor data is reduced to problem (9). Note that (9) is convex but non-differentiable due to the nuclear norm. Hence, we adopt the alternating methods of multipliers method (ADMM) which allows for closed-form solutions in each step and ensures convergence to the global minimum in a finite number of iterations [35]. Toward this, we firstly introduce the auxiliary variables, and rewire (9) as the following equivalent splitting version:

$$\begin{aligned} \min_{\mathbf{X}} \quad & \frac{1}{2} \|\mathbf{M} \odot (\mathbf{X} - \mathbf{Z})\|_2^2 + \frac{\alpha}{2} \mathbf{x}^T \mathbf{L}^k \mathbf{x} + \mu \|\mathbf{Y}\|_* \\ \text{s.t. } \quad & \mathbf{X} = \mathbf{Y}, \end{aligned} \quad (11)$$

The corresponding augmented Lagrangian is written by adding additional quadratic terms to ensure the attainment of the equality constraints:

$$\begin{aligned} L(\mathbf{X}, \mathbf{Y}, \mathbf{P}) = \quad & \frac{1}{2} \|\mathbf{M} \odot (\mathbf{X} - \mathbf{Z})\|_2^2 + \frac{\alpha}{2} \mathbf{x}^T \mathbf{L}^{\otimes} \mathbf{x} + \mu \|\mathbf{Y}\|_* \\ & + \text{tr}(\mathbf{P}^T (\mathbf{X} - \mathbf{Y})) + \rho \|\mathbf{X} - \mathbf{Y}\|_F^2, \end{aligned} \quad (12)$$

where $\mathbf{P}$ is the Lagrange multipliers (dual variable) and ρ is the penalty parameter. Then, ADMM solve (12) by following the steps where the primal variable $\mathbf{X}$ is updated at the k^{th} iteration by solving the sub-problem:

$$\begin{aligned} \mathbf{X}^{(k+1)} = \arg \min_{\mathbf{X}} \quad & g(\mathbf{X}) = \frac{1}{2} \|\mathbf{M} \odot (\mathbf{X} - \mathbf{Z})\|_2^2 + \frac{\alpha}{2} \mathbf{x}^T \mathbf{L}^{\otimes} \mathbf{x} \\ & + \frac{\rho}{2} \|\mathbf{X} - \mathbf{Y}^{(k)}\|_F^2 + \frac{\mathbf{P}^{(k)}}{\rho}. \end{aligned} \quad (13)$$

Note that function $g(\mathbf{X})$ is a quadratic function, therefore (13) admits closed form solution. To proceed, we first reformulate the smoothness term in the objective. By exploiting the fact that for any given matrix $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ and $\mathbf{X}$, the equation $\mathbf{AXB} = \mathbf{C}$ is equivalent to $(\mathbf{B}^T \otimes \mathbf{A})\text{vec}(\mathbf{X}) = \text{vec}(\mathbf{C})$ [36], the smoothness term with the Laplacian of the Kronecker product graph can be written as

$$\begin{aligned} \mathbf{x}^T \mathbf{L}^{\otimes} \mathbf{x} &= \mathbf{x}^T (\mathbf{D}_1 \otimes \mathbf{D}_2 - \mathbf{W}_1 \otimes \mathbf{W}_2) \mathbf{x} \\ &= \mathbf{x}^T (\mathbf{D}_1 \otimes \mathbf{D}_2 - (\mathbf{D}_1 - \mathbf{L}_1) \otimes (\mathbf{D}_2 - \mathbf{L}_2)) \mathbf{x} \\ &= \mathbf{x}^T (\mathbf{L}_1 \otimes \mathbf{D}_2 + \mathbf{D}_1 \otimes \mathbf{L}_2 - \mathbf{L}_1 \otimes \mathbf{L}_2) \mathbf{x} \\ &= \mathbf{x}^T \text{vec}(\mathbf{D}_2 \mathbf{X}^T \mathbf{L}_1 + \mathbf{L}_2 \mathbf{X}^T \mathbf{D}_1 - \mathbf{L}_2 \mathbf{X}^T \mathbf{L}_1) \\ &= \text{tr}(\mathbf{X}(\mathbf{D}_2 \mathbf{X}^T \mathbf{L}_1 + \mathbf{L}_2 \mathbf{X}^T \mathbf{D}_1 - \mathbf{L}_2 \mathbf{X}^T \mathbf{L}_1)), \end{aligned} \quad (14)$$

where the last equation holds by the trace property. With this arrangement, the gradient of $g(\mathbf{X})$ can be given as

$$\begin{aligned} \nabla g(\mathbf{X}) = \quad & \mathbf{M} \odot \mathbf{X} + \alpha (\mathbf{L}_1 \mathbf{X} \mathbf{D}_2 + \alpha \mathbf{D}_1 \mathbf{X} \mathbf{L}_2 - \mathbf{L}_1 \mathbf{X} \mathbf{L}_2) \\ & + \rho (\mathbf{X} - \mathbf{Y}^{(k)}) - \mathbf{Z} + \mathbf{P}^{(k)}. \end{aligned} \quad (15)$$

Then we can obtain the optimal solution by setting $\nabla g(\mathbf{X}) = 0$. However, it requires computing the inversion of a matrix of size $NT \times NT$, which does not scale favorably. Therefore, we consider minimizing $g(\mathbf{X})$ directly by applying conjugate gradient (CG) iterations [37]

$$\mathbf{X}_{\tau+1} = \mathbf{X}_{\tau} + \eta \Delta \mathbf{X}_{\tau}, \quad (16)$$

where $\Delta \mathbf{X}_{\tau} = -\nabla g(\mathbf{X}_{\tau}) + \theta \Delta \mathbf{X}_{\tau-1}$ is the search direction at τ^{th} iterations, $\theta = \frac{\|\nabla g(\mathbf{X}_{\tau})\|_F^2}{\|\nabla g(\mathbf{X}_{\tau-1})\|_F^2}$, and η is the step size determined e.g., by the line minimization rule.

The update of $\mathbf{Y}^{(k+1)}$ has a closed form solution corresponding to the following problem:

$$\begin{aligned} \mathbf{Y}^{(k+1)} &= \arg \min_{\mathbf{Y}} \quad \frac{\rho}{2} \|\mathbf{X}^{(k+1)} - \mathbf{Y} + \frac{\mathbf{P}^{(k)}}{\rho}\|_F^2 + \mu \|\mathbf{Y}\|_* \\ &= \text{prox}_{\frac{\mu}{\rho} \|\mathbf{Y}\|_*} (\mathbf{X}^{(k+1)} + \frac{\mathbf{P}^{(k)}}{\rho}), \end{aligned} \quad (17)$$

where $\text{prox}_h(\tilde{\mathbf{Y}}) = \arg \min_{\mathbf{Y}} h(\mathbf{Y}) + \frac{1}{2} \|\mathbf{Y} - \tilde{\mathbf{Y}}\|_F^2$ is the proximity operator of the convex function h . Let $\tilde{\mathbf{Y}} = \mathbf{Q} \Sigma \mathbf{V}^T$ denotes the singular value decomposition of matrix $\tilde{\mathbf{Y}}$, and then the singular value soft thresholding is used to update $\mathbf{Y}^{(k+1)}$

$$\mathbf{Y}^{(k+1)} = \mathbf{Q} \Omega_{\frac{\mu}{\rho}}(\Sigma) \mathbf{V}^T, \quad (18)$$

where $\Omega_{\tau}(a) = \text{sgn}(a) \max(|a| - \tau, 0)$ is defined to be the element-wise thresholding operator.

Finally, the update of dual variable $\mathbf{P}^{(k+1)}$ is performed by

$$\mathbf{P}^{(k+1)} = \mathbf{P}^{(k)} + \rho (\mathbf{X}^{(k+1)} - \mathbf{Y}^{(k+1)}). \quad (19)$$

When updating $\mathbf{X}$, the computational cost of (16) is dominated by computing the product $\mathbf{L}_1 \mathbf{X} \mathbf{L}_2$, which is proportional to $\mathcal{O}(\kappa_{nnz}(\mathbf{L}_1)T + \kappa_{nnz}(\mathbf{L}_2)N)$, where $\kappa_{nnz}(\cdot)$ counts the number of non-zero elements of a matrix. Note that either the learned spatial graph or the pre-defined time-series graph are both sparse, which renders the complexity of (16) scaling linearly in T . For the update of $\mathbf{Y}$, the computation of nuclear norm costs $\mathcal{O}(N^2T)$ when $N < T$. The complexity of (19) is $\mathcal{O}(N^2T)$. Thus, the dominant cost comes from updating $\mathbf{Y}$ during a single iteration, and the overall computation complexity per ADMM iteration is $\mathcal{O}(N^2T)$.

B. Spatial Graph Learning

On the other hand, once we complete the sensor data matrix, we can find the underlying spatial sensor graph by solving

$$\min_{\mathbf{L}_1 \in \mathcal{L}} \quad \frac{\alpha}{2} \mathbf{x}^T \mathbf{L}^{\otimes} \mathbf{x} - \beta_1 \text{tr}(\mathbf{1}^T \log(\text{diag}(\mathbf{L}_1))) + \beta_2 \|\mathbf{L}_1\|_{1, \text{off}}. \quad (20)$$

The log-barrier term acts on the node degree of the spatial graph, which guarantees the solution to be a connected graph. Then, even for the sparsest graph, each sensor can still relates with its first nearest neighbor, thereby avoiding a trivial solution and promoting the connectivity of the learned graph.

In the next, we simplify the optimization problem (20) to efficiently learn the spatial sensor graph. Given that optimizing with the weighted adjacency matrix will be much simpler than with graph Laplacian matrix since a valid adjacency matrix only needs to be satisfied the constraints of symmetry and non-negativity, i.e., $W_{i,j} = W_{j,i} \geq 0$, $\forall i \neq j$ and $W_{i,i} = 0$. Toward this, let us further decouple the smoothness term as

$$\begin{aligned} \mathbf{x}^T \mathbf{L} \otimes \mathbf{x} &= \mathbf{x}^T (\text{diag}(\mathbf{W}_1 \mathbf{1}_N) \otimes \text{diag}(\mathbf{W}_2 \mathbf{1}_T) - \mathbf{W}_1 \otimes \mathbf{W}_2) \mathbf{x} \\ &= \mathbf{x}^T (\text{diag}(\mathbf{x}) (\mathbf{W}_1 \otimes \mathbf{W}_2) \mathbf{1}_{NT} - (\mathbf{W}_1 \otimes \mathbf{W}_2) \mathbf{x}) \\ &= \bar{\mathbf{x}}^T \text{vec}(\mathbf{W}_2 \mathbf{1}_T \mathbf{1}_N^T \mathbf{W}_1) - \mathbf{x}^T \text{vec}(\mathbf{W}_2 \mathbf{X}^T \mathbf{W}_1) \\ &= \text{tr}(\bar{\mathbf{X}} \mathbf{W}_2 \mathbf{1}_T \mathbf{1}_N^T \mathbf{W}_1 - \mathbf{X} \mathbf{W}_2 \mathbf{X}^T \mathbf{W}_1) \\ &= \text{tr}(\mathbf{W}_1 (\bar{\mathbf{X}} \mathbf{W}_2 \mathbf{1}_T \mathbf{1}_N^T - \mathbf{X} \mathbf{W}_2 \mathbf{X}^T)), \end{aligned} \quad (21)$$

where we have $\bar{\mathbf{x}} = (\mathbf{x}^T \text{diag}(\mathbf{x}))^T = \text{vec}(\bar{\mathbf{X}})$. To proceed, we can rewrite (20) as

$$\min_{\mathbf{W}_1 \in \mathcal{W}} \frac{\alpha}{2} \text{tr}(\mathbf{W}_1 \mathbf{S}) - \beta_1 \mathbf{1}_N^T \log(\mathbf{W}_1 \mathbf{1}_N) + \beta_2 \|\mathbf{W}_1\|_F^2, \quad (22)$$

where $\mathcal{W} = \{\mathbb{R}_+^{N \times N}, \mathbf{W}_1 = \mathbf{W}_1^T, \text{diag}(\mathbf{W}_1) = \mathbf{0}_N\}$ is the set of valid weighted adjacency matrices, and matrix $\mathbf{S} = \bar{\mathbf{X}} \mathbf{W}_2 \mathbf{1}_T \mathbf{1}_N^T - \mathbf{X} \mathbf{W}_2 \mathbf{X}^T$. In fact, since adjacency matrix is symmetric and zero diagonal, thus only $N(N-1)/2$ variables on the lower or upper triangular part are needed to be considered for optimization rather than the whole matrix. Let us define the duplication matrix as $\mathbf{B} \in \mathbb{R}^{N^2 \times N(N-1)/2}$ which duplicates the elements of the lower triangular part $\mathbf{w} \in \mathbb{R}^{N(N-1)/2}$ to produce a vectorized version of $\mathbf{W}$ such that $\text{vec}(\mathbf{W}) = \mathbf{B}\mathbf{w}$ [36]. In order to make the problem tractable, we simplify (22) as

$$\min_{\mathbf{w} \geq 0} \frac{\alpha}{2} \mathbf{s}^T \mathbf{B}\mathbf{w} - \beta_1 \mathbf{1}_N^T \log(\mathbf{R}\mathbf{w}) + 2\beta_2 \|\mathbf{w}\|_2^2, \quad (23)$$

where $\mathbf{s} = \text{vec}(\mathbf{S})$, and matrix $\mathbf{R} \in \mathbb{R}^{N \times N(N-1)/2}$ handles the equality $\mathbf{W}_1 \mathbf{1}_N = \mathbf{R}\mathbf{w}$. Here, the symmetry constraint does not need to be imposed as a constraint. Problem (23) can be split into a differentiable part and two proximable parts for which the proximal operators can be efficiently computed. Thus, we solve (23) by using the primal-dual splitting (PDS) method [38] with small modifications. The computational complexity of this algorithm is $\mathcal{O}(N^2)$ per iteration, which scales quadratically with the number of graph vertices.

As the optimization problem in the proposed graph learning process is convex and lower-Semicontinuous, convergence is assured in this algorithm. Once the optimal edge weight vector $\hat{\mathbf{w}}$ are obtained, we then organize it into the standard form of the Laplacian matrix for completing the sensor data in the next iteration. To be noted that, contrary to the existing graph learning approaches, the innovation of our method is that we consider the temporal structure of the sensor data when we learn an underlying graph that best captures the spatial correlations, which thus improves the quality of the learned graph. The overall algorithm for the graph learning-based missing data recovery is summarized in **Algorithm 1**.

IV. NUMERICAL EXPERIMENTS

A. Data Description

In this section, we assess the performance of our proposed algorithm on two distinct time-series datasets: the climate

Algorithm 1 Graph Learning-Based Data Imputation

Input: Sensor data measurements $\mathbf{Z}$, indicator matrix $\mathbf{M}$, Laplacian matrix $\mathbf{L}_2$ of $\mathcal{G}_2$, graph product structure $\diamond$, regularization parameters α , μ , β_1 , and β_2 .

Output: Recovered sensor data $\hat{\mathbf{X}}$.

- 1 Initialize $\mathbf{w}(0) = \mathbf{0}$, $\hat{\mathbf{X}}(0) = \mathbf{Z}$;
- 2 **while** iterates not converge **do**
- 3 Duplicate the updated edge vector $\mathbf{w}(i)$ to the Laplacian matrix $\mathbf{L}_1(i)$;
- 4 Update $\hat{\mathbf{X}}(i)$ from (11) using ADMM;
- 5 Update the matrix $\mathbf{S}(i)$;
- 6 Update edge vector $\mathbf{w}(i)$ from (23) using PDS;
- 7 $i = i + 1$
- 8 **end**

dataset provided in the NCEP Reanalysis data [39], and the traffic-state data from the Seattle Inductive Loop Detector Dataset [40]. The two datasets are detailed as follows.

Climate Dataset: This dataset consists of the temperature data recorded in weather stations distributed over 45 states in the U.S. Daily average of the temperature measurements is calculated based on the time period 2000-2015, thus a total of 5844 snapshots for each of the $N = 45$ states. We only choose the first $T = 1000$ samples for the experiment. The average and standard deviation of the selected data are 285.427 K and 10.498 K, respectively. The temperature readings at adjacent states tend to have similar values. However, there are lower average temperature values in the Rocky Mountains than its geographical neighborhoods due to the significant terrain variations, and thus this region is the most non-smooth parts on a geographical distance-based graph.

Traffic-State Dataset: Consider the Seattle Inductive Loop Detector Dataset, which comprises the spatiotemporal speed information of the freeway system across the Seattle area in 2015. There is a total of 323 inductive loop detectors deployed on freeways to obtain the speed information. The time interval of the dataset is 5-minute and thus providing in total 105,120 time-steps. The dimension of the selected speed data is 323×1000 . The average and standard deviation of the selected data are 62.09 miles/h and 5.73 miles/h respectively. The traffic network describing the Loop connections is also provided by the source in the form of a binary adjacency matrix.

B. Evaluation Criteria

To evaluate the imputation performance, we adopt three criteria including root mean square error (RMSE), the mean absolute error (MAE) and mean absolute percentage error (MAPE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2},$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |x_i - \hat{x}_i|,$$

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{x_i - \hat{x}_i}{x_i} \right|,$$

where n is the total number of missing sensor data, $\hat{x}_i$ is the estimated data which corresponds to the ground truth x_i . The MAE usually handles more uniformly the imputation errors, while RMSE is more sensitive to large imputation errors. We use MAPE to compare the performance of the imputation methods for two distinct types of data.

C. Experiment Details

Firstly, the values in each dataset are scaled to the range of $[0, 1]$ before applying the data imputation, and the output of each method is re-scaled to the original values range. The evaluation metrics are therefore calculated using the original and the imputed re-scaled data. As regularization parameters for (23) we set $\beta_1 = \beta_2 = 1$ and input vector $\alpha \mathbf{s}^T \mathbf{B} = \delta \mathbf{s}^T \mathbf{B}$ with δ a tuning parameter. Then for each dataset, we empirically tuned the parameters α , μ and δ , as well as the edge weight c of line graph by grid searches over different sets of values and selected the combination of parameters with the best MAE result.

To simulate the missing patterns, we introduce missing values into the data matrix by randomly removing values based on a certain missing ratio. Usually, the absence of data will be classified into three types of missing patterns: Missing Completely at Random (MCAR), Missing at Random (MAR), and Missing Not at Random (MNAR) [5].

The MCAR is the most common missing pattern due to temporary communication failure or other random events. Thus, the distribution of missing values is random and independent of the missing values themselves. Fig. 2(a) presents an intuitive example of the MCAR pattern. The MAR is more complicated and often caused by the sensor node break down. The missing values of a sensor node are related to the state of its adjacent sensor nodes. For example, speed data for one particular road segment may be lost during one day. Thus, the missingness may appear as a successive missing in several periods at one sensor (as Fig. 2(b)). Moreover, the missing mechanism of MNAR is non-random and the unobserved values are dependent on the variable itself.

In our experiment, we design MCAR and MAR missing patterns for generating the simulation data. For the MCAR scenario, given the missing ratio p , we replaces $p \times (NT)$ entries of a all-ones indicator matrix $\mathbf{M} \in \mathbb{R}^{N \times T}$ with zeros at random and obtain the incomplete data matrix $\mathbf{Z} = \mathbf{M} \odot \mathbf{X}$. As for MAR, we first set the window size t of the successive missing and generate the indicator matrix $\mathbf{M} \in \mathbb{R}^{N \times \frac{T}{t}}$ based on the missing ratio. Then, the incomplete data matrix can be prescribed by $\mathbf{Z} = (\mathbf{M} \otimes \mathbf{1}^T) \odot \mathbf{X}$. In this paper, the window size t is set to 10 for the sample generation. Then, we ran simulations for 20 independently random sampling patterns and averaged the results.

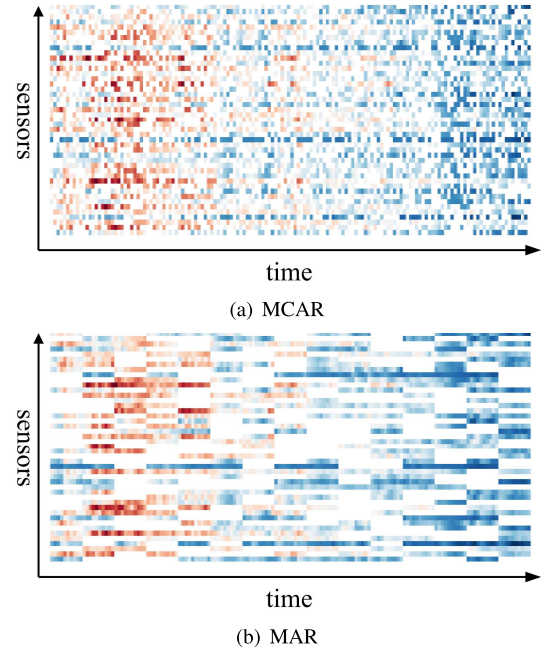


Fig. 2. Missing patterns of sensor data. (a) Data are missing completely at random (MCAR). (b) Data are missing at random (MAR).

D. Baselines

We compare our proposed approach against five commonly used imputation methods in the sensor network. The details of these methods are outlined below:

KNN [41]: The K-nearest neighbors (**KNN**) method imputes the missing sensor data $X_{n,t}$ by using its k nearest neighbors that have available data in t^{th} time step with weights based on the Euclidean distance to n^{th} sensor.

Spectral [42]: The **Spectral** regularization algorithm uses the nuclear norm as a regularizer and imputes missing sensor data based on matrix completion by iterative soft thresholding of SVD decomposition.

IterativeImputer [43]: The **IterativeImputer** imputes the missing values by modeling each feature with missing values as a function of other features in a round-robin fashion.

MF [16]: The Matrix Fraction (**MF**) method imputes missing sensor data by directly factorizing the incomplete data matrix into two low-rank latent factor matrices for sensor network and time steps respectively.

MC-Graph [28]: The matrix completion on graphs recover the missing data by using the nuclear norm and individual graphs, i.e. row and column graphs.

For the baselines including **KNN**, **Spectral**, and **MF**, we adopt the implementation provided in the *fancyimpute*¹ package. We use the *scikit-learn*² package for the implementation of the **IterativeImputer**. To provide a fair comparison, the hyper-parameters that gave the best results are chosen for all methods. For **KNN**, we found $k = 20$ works the best. For **IterativeImputer**, we use the default *BayesianRidge* estimator. For **MC-Graph**, we use the pre-defined spatial graph as the column graph and the line graph as the row graph.

¹<https://github.com/jskandr/fancyimpute>

²<https://github.com/scikit-learn/scikit-learn>

TABLE I
PERFORMANCE OF DIFFERENT PRODUCT GRAPHS IN THE TEMPERATURE DATA

Missing pattern	Ratio	Cartesian + graph learning		Cartesian + fixed graph		Strong + graph learning		Strong + fixed graph	
		MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
MCAR	p=90%	2.339	3.092	2.841	3.840	2.400	3.163	2.918	3.946
	p=70%	1.414	1.985	1.505	2.152	1.434	2.014	1.562	2.229
	p=50%	0.950	1.405	0.975	1.460	0.955	1.410	0.993	1.489
	p=30%	0.676	1.039	0.699	1.074	0.679	1.044	0.703	1.080
MAR	p=90%	3.092	4.109	3.467	4.689	3.055	4.041	3.486	4.704
	p=70%	1.722	2.412	1.913	2.763	1.718	2.436	1.949	2.804
	p=50%	1.070	1.578	1.087	1.636	1.095	1.609	1.104	1.660
	p=30%	0.707	1.078	0.744	1.149	0.712	1.085	0.750	1.159

TABLE II
PERFORMANCE OF DIFFERENT PRODUCT GRAPHS IN THE TRAFFIC DATA

Missing pattern	Ratio	Cartesian + graph learning		Cartesian + fixed graph		Strong + graph learning		Strong + fixed graph	
		MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
MCAR	p=90%	2.193	3.287	2.292	3.461	2.143	3.243	2.310	3.498
	p=70%	1.884	2.817	1.997	2.996	1.908	2.844	2.006	3.016
	p=50%	1.725	2.601	1.862	2.772	1.750	2.632	1.865	2.782
	p=30%	1.640	2.476	1.768	2.621	1.663	2.501	1.769	2.625
MAR	p=90%	2.589	4.171	2.661	4.086	2.550	4.108	2.691	4.123
	p=70%	2.185	3.483	2.276	3.522	2.151	3.418	2.333	3.585
	p=50%	1.955	3.069	2.054	3.151	1.966	3.103	2.118	3.226
	p=30%	1.810	2.829	1.903	2.934	1.828	2.844	1.965	3.010

E. Results

In this subsection, we present the imputation results of our proposed method. Firstly, we analyze the effect of the type of product graph and the graph learning of our algorithm. Secondly, we focus on the comparison of the performance between our algorithm and baselines.

Setting the missing ratio $p = 30\%, 50\%, 70\%$, and 90% , we firstly investigate the imputation performance of different type of product graph under different missing ratios. The Kronecker graph is ignored in the experiment since this architecture is included in the Strong product graph. As a comparison, we consider the imputation working with a fixed-graph in the spatial domain. For temperature data, a k nearest neighbors graph based on the Euclidean distances between stations are created with nonzero edge weights given by the Gaussian kernel function. We heuristically let $k = 5$ and σ as the mean of the distances between all stations. For traffic data, we take the traffic network structure as a graph and further weighted the graph by setting the edge weights as $W(i, j) = e^{-\|x_i - x_j\|^2 / 10}$, where x_i and x_j are speed time series with non-missing values corresponding to i^{th} sensor and j^{th} sensor, respectively. Furthermore, we set the constant $c = 0.5$ for all trials. We use $\alpha = 10^{-3}$, $\mu = 0.05$ and $\delta = 0.3$ for temperature data, while $\alpha = 10^{-4}$, $\mu = 0.1$ and $\delta = 0.9$ for traffic data.

Table I and Table II show the comparison between different product graphs with $T = 1000$. As expected, in these two datasets, the imputation performance is improved with the decreasing missing ratio. We can observe that the imputation with the graph learning step achieves the lowest MAE and RMSE compared to the imputation with the fixed graph. An intuitive explanation is that the pre-defined graph is not

appropriate for describing the spatial correlation and thus leads to poor data imputation performance, but the graph learning aid capturing the intrinsic dependencies between sensors and reflects in a better data imputation performance. This corroborates that our strategy, which estimates both the missing data and the underlying spatial graph simultaneously, can enhance the data imputation performance. Interestingly, we can also observe that the algorithm with Strong product graph is performed well in the high missing ratio, but have poor imputation performance in the lower missing ratio compared with the Cartesian product graph, although its graph structure encodes more spatiotemporal correlations. We hypothesize that the redundant edges of the Strong product graph may introduce the noise when modeling the spatiotemporal correlations among data, and further affect the imputation performance. On the other hand, when the missing ratio decreases, more connections in the graph may provide richer information about the underlying spatiotemporal correlations and thus play an active role in data recovery, especially in the MAR missing pattern.

Next, the proposed method is compared to other imputation methods with $T = 1000$ under different missing ratios. Table III shows the results of all methods under different missing ratios. The edge weight c of the line graph is set to 0.5 for the temperature data, and 5 for the traffic data. As observed from the results, the imputation error of all methods decreases with the decreasing missing ratio, and the performance of the proposed method is always superior compared to that of other methods. When the missing ratio is 70% in MACR missing pattern, the proposed method achieves 1.99 and 2.76 RMSE in the temperature and speed data respectively. Without using the graph structure, IterativeImputer achieves the best result in

TABLE III
PERFORMANCE OF MISSING DATA IMPUTATION UNDER THE MACR PATTERN

Dataset	Ratio	Our		MC-Graph		KNN		Spectral		IterativeImputer		MF	
		RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Temperature data (K)	p=90%	3.09	2.34	3.29	2.51	9.52	5.19	5.63	3.75	8.50	6.81	6.43	5.06
	p=70%	1.99	1.41	2.09	1.47	4.06	2.91	2.88	2.06	2.74	1.94	3.71	2.65
	p=50%	1.41	0.95	1.43	0.97	2.37	1.73	1.71	1.24	1.63	1.12	2.47	1.75
Traffic data (miles/h)	p=90%	3.01	2.02	3.15	2.17	4.81	2.9	5.63	3.75	4.54	2.89	4.51	2.84
	p=70%	2.76	1.81	2.77	1.93	3.40	2.02	2.98	3.19	3.51	2.26	3.60	2.23
	p=50%	2.64	1.73	2.62	1.73	3.04	1.88	1.71	2.84	3.07	2.02	3.34	2.10

the temperature data and the RMSE is 2.74, and in the traffic data, the best method Spectral has 2.98 RMSE. The dominance of our method is even more apparent when the missing ratio is 90%, achieving only 3.09 RMSE and 2.34 MAE in the temperature data, and 3.01 RMSE and 2.02 MAE in the traffic data. Additionally, the proposed method provides more stable results when the missing ratio is high, while the imputation errors of other models are increasing. The improvement of our method compared with KNN and IterativeImputer is owing to the exploitation of the low-rank assumption. The improvement of our method over Spectral and MF comes from the proper modeling of the temporal and spatial correlation inherent in sensor data. Moreover, the proposed method can learn the latent correlations among sensors and thus outperforms the other methods even when the number of remaining values is small. The ensemble of spatial and temporal correlations significantly improves the accuracy of the data imputation. As can be seen from the results, given the predefined spatial graph and temporal graph, the MC-Graph method can also leverage the spatial and temporal correlations for imputation, and achieve better results compared with other methods. However, MC-Graph does not perform well than our method under different missing ratios. As the proposed method differs from the MC-Graph method mainly in the use of spatial graph learning, the results justify the positive impact brought by spatial graph learning. Moreover, the MC-graph have a regularization parameter to tune the temporal smoothness in the objective function. However, our proposed method can also achieve the same effect by optimally choosing the edge weight c in the line graph. In fact, the matrix completion method with individual graphs, which uses the spatial smoothness term with the temporal smoothness term separately, can be interpreted as a special case of our method working on the Cartesian structured graph. But our method can also address the data imputation problem on the Strong product graphs and Kronecker graph. Therefore, our method can be employed efficiently imputing missing sensor measurements.

As shown in Table IV, the imputation errors of all methods in the MAR missing pattern are higher than the results in the MCAR missing pattern. This is mainly because the MAR missing pattern is more intractable as many sequential points are missing at each sensor. The available information of the sensor measurements are reducing as the missing ratio increase. However, our proposed method still possesses the lowest errors among all missing ratios. Although the other method can achieve good performance with a low missing ratio, their performance decreases dramatically when the miss-

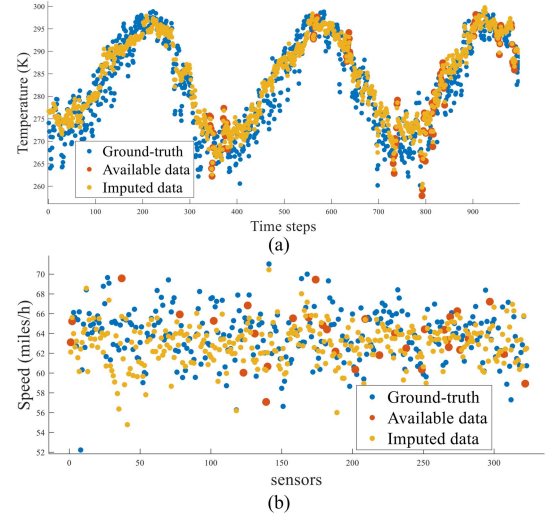


Fig. 3. Imputed data of (a) the Colorado state in the temperature dataset and (b) the 30th time step for all sensors in traffic dataset.

ing ratio is greater. Additionally, other solutions may fail to capture the inherent coupling between space domains. Fig. 3 depicts the ground truth, along with the imputed values in which 90% of the values are missing under MAR. These plots further corroborate the effectiveness of our method in imputing the missing data. Fig. 3(a) plots $T = 1000$ temperature values in the Colorado state. Since the temperature values around the Rocky Mountains region may be the most non-smooth parts over the distance-based graph, the imputation around this region will be difficult. However, our method still fits well with the ground truth with only 10% data available. All results indicate that the learned topology information can successfully capture the underlying spatial correlations among sensors. Thus, our method can take much valuable information from multiple views in the imputation phase, so it can keep stable performance.

To evaluate the performance of our algorithm for two distinct types of data, in Fig. 4 we show the comparison of MAPE for the same simulation setting. The errors in the temperature dataset are lower than the results in the traffic speed dataset. With 10% sensor data available under MAR pattern the MAPE is only 1.08% for temperature data, and 4.92% for traffic speed data. This is because the temperature data may be highly smooth with respect to the ST graph, which follows the assumption that the spatiotemporal sensor data is smooth, thus providing better imputation performance. On the other hand, the traffic data are not as smooth as temperature, leading to a larger imputation error.

TABLE IV
PERFORMANCE OF MISSING DATA IMPUTATION UNDER THE MAR PATTERN

Dataset	Ratio	Our		MC-Graph		KNN		Spectral		IterativeImputer		MF	
		RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Temperature data (K)	p=90%	4.04	3.06	4.43	3.33	10.74	5.97	4.92	3.74	7.03	5.08	7.06	5.40
	p=70%	2.44	1.72	2.61	1.92	4.43	3.18	3.33	2.43	3.48	2.45	4.28	3.10
	p=50%	1.58	1.07	1.64	1.1	3.25	2.35	2.08	1.44	1.87	1.21	2.77	2.05
Traffic data (miles/h)	p=90%	3.99	2.20	4.13	2.61	4.46	3.06	5.48	4.12	4.90	3.17	4.72	2.98
	p=70%	3.31	2.09	3.46	2.17	4.27	2.61	4.60	2.92	4.33	2.87	3.89	2.34
	p=50%	3.02	1.95	3.15	1.97	4.04	2.47	3.72	2.33	4.15	2.70	3.47	2.16

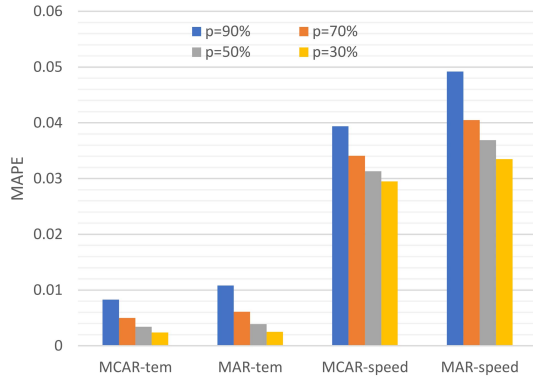


Fig. 4. MAPE for data imputation in temperature dataset and traffic dataset under different missing pattern.

TABLE V
COMPUTATIONAL TIME OF PROPOSED METHOD WITH DIFFERENT MISSING RATIOS UNDER MAR MISSING PATTERN

	p=30%	P=50%	P=70%	P=90%
Temperatue data	0.014	0.031	0.045	0.063
Traffic data	0.818	0.866	0.932	0.943

Finally, we compare the computational time of our methods in the MAR missing pattern, and the results are shown in Table V. We can find that the computational time gradually increases with the increasing missing ratio. In our proposed method, the computational time comes from the iteration between the graph learning and the data imputation. With I_x iterations for the ADMM algorithm, the computation complexity for data imputation scales $\mathcal{O}(I_x N^2 T)$. The graph learning has the computational complexity scaling with $\mathcal{O}(I_w N^2)$, where I_w is the number of iterations for PDS algorithm. Assuming that the overall number of iterations for Algorithm 1 is I_o , then total complexity of the graph learning-based data imputation method costs $\mathcal{O}(I_o(I_x N^2 T + I_w N^2))$. Thus, since more sensor nodes are in the traffic data, the computational times are much higher than those in the temperature data.

V. CONCLUSION

In this paper, we propose a graph learning-based data recovery approach to impute missing values in the spatiotemporal sensor data. The key insight of this paper is to interpret the ST correlations as the edge weights of the product graph. Then the data imputation task can naturally be formulated as the data recovery problem on the graph. Considering that the

geographical sensor graph may not perfectly capture the real complex spatial dependencies between sensor nodes, we aim to learn the underlying relations from data to improve imputation performance. The experiment results on two real data sets corroborate that the method proposed can impute the missing data efficiently.

For future research, we will exploit the low-dimensional structure of the data on the product graph to study the sensor data imputation problem. It is straightforward to organize the spatiotemporal sensor data matrix into a higher dimension tensor. For instance, the sensor data characterized by the dimensions of sensors, date, and time interval one day can be organized as a 3rd tensor (sensor $\times$ day $\times$ time interval one day). Thus, the sensor data lay on a product of three graphs corresponds to three dimensions, leading to a low-dimensional structure of data.

REFERENCES

- [1] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Generat. Comput. Syst.*, vol. 29, no. 7, pp. 1645–1660, Feb. 2013.
- [2] L. U. Khan, I. Yaqoob, N. H. Tran, S. M. A. Kazmi, T. N. Dang, and C. S. Hong, "Edge-computing-enabled smart cities: A comprehensive survey," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10200–10232, Oct. 2020.
- [3] S. Gahlot, S. R. N. Reddy, and D. Kumar, "Review of smart health monitoring approaches with survey analysis and proposed framework," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2116–2127, Apr. 2019.
- [4] J. Yick, B. Mukherjee, and D. Ghosal, "Wireless sensor network survey," *Comput. Netw.*, vol. 52, no. 12, pp. 2292–2330, Aug. 2008.
- [5] J. Du, M. Hu, and W. Zhang, "Missing data problem in the monitoring system: A review," *IEEE Sensors J.*, vol. 20, no. 23, pp. 13984–13998, Dec. 2020.
- [6] J. Ploennigs, V. Vasyutynskyy, and K. Kabitzsch, "Comparative study of energy-efficient sampling approaches for wireless control networks," *IEEE Trans. Ind. Inf.*, vol. 6, no. 3, pp. 416–424, Aug. 2010.
- [7] F. Wu, K. Yang, and Z. Yang, "Compressed acquisition and denoising recovery of EMGdi signal in WSNs and IoT," *IEEE Trans. Ind. Inf.*, vol. 14, no. 5, pp. 2210–2219, May 2018.
- [8] K. Kiani and K. Saleem, "K-nearest temperature trends: A method for weather temperature data imputation," in *Proc. Int. Conf. Inf. Syst. Data Mining (ICISDM)*, 2017, pp. 23–27.
- [9] K. Zhang, R. Gonzalez, B. Huang, and G. Ji, "Expectation-maximization approach to fault diagnosis with missing data," *IEEE Trans. Ind. Electron.*, vol. 62, no. 2, pp. 1231–1240, Feb. 2015.
- [10] L. Gruenwald and M. Halatchev, "Estimating missing values in related sensor data streams," in *Proc. 11th Int. Conf. Manag. Data (COMADO)*, 2005, pp. 83–94.
- [11] X. Yi, Y. Zheng, J. Zhang, and T. Li, "ST-MVL: Filling missing values in geo-sensory time series data," in *Proc. 25th Int. Joint Conf. Artif. Intell.*, 2016, pp. 1–7.
- [12] P. Gao, M. Wang, S. G. Ghiocel, J. H. Chow, B. Fardanesh, and G. Stefopoulos, "Missing data recovery by exploiting low-dimensionality in power system synchrophasor measurements," *IEEE Trans. Power Syst.*, vol. 31, no. 2, pp. 1006–1013, Mar. 2016.

- [13] X. Chen, Z. Wei, Z. Li, J. Liang, Y. Cai, and B. Zhang, "Ensemble correlation-based low-rank matrix completion with applications to traffic data imputation," *Knowl.-Based Syst.*, vol. 132, pp. 249–262, Sep. 2017.
- [14] B. Ran, H. Tan, J. Feng, Y. Liu, and W. Wang, "Traffic speed data imputation method based on tensor completion," *Comput. Intell. Neurosci.*, vol. 2015, pp. 1–9, Mar. 2015.
- [15] M. T. Asif, N. Mitrovic, J. Dauwels, and P. Jaillet, "Matrix and tensor based methods for missing data estimation in large traffic networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 7, pp. 1816–1825, Jul. 2016.
- [16] B. Fekade, T. Maksymyuk, M. Kyryk, and M. Jo, "Probabilistic recovery of incomplete sensed data in IoT," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2282–2292, Aug. 2018.
- [17] A. Ortega, P. Frossard, J. Kovačević, J. M. F. Moura, and P. Vanderghenst, "Graph signal processing: Overview, challenges, and applications," *Proc. IEEE*, vol. 106, no. 5, pp. 808–828, May 2018.
- [18] H. E. Egilmez and A. Ortega, "Spectral anomaly detection using graph-based filtering for wireless sensor networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, May 2014, pp. 1085–1089.
- [19] Z. Xiao, H. Fang, and X. Wang, "Nonlinear polynomial graph filter for anomalous IoT sensor detection and localization," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4839–4848, Jun. 2020.
- [20] A. Mazarguil, L. Oudre, and N. Vayatis, "Localized interpolation for graph signals," in *Proc. 28th Eur. Signal Process. Conf. (EUSIPCO)*, Jan. 2021, pp. 2160–2164.
- [21] M. Kaneko, G. Cheung, W.-T. Su, and C.-W. Lin, "Graph-based joint signal/power restoration for energy harvesting wireless sensor networks," Apr. 2017, *arXiv:1704.06012*. [Online]. Available: <http://arxiv.org/abs/1704.06012>
- [22] A. Sakiyama, Y. Tanaka, T. Tanaka, and A. Ortega, "Efficient sensor position selection using graph signal sampling theory," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Mar. 2016, pp. 6225–6229.
- [23] S. Liu *et al.*, "Handling missing sensors in topology-aware IoT applications with gated graph neural network," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 4, no. 3, pp. 1–31, Sep. 2020.
- [24] G. Mateos, S. Segarra, A. G. Marques, and A. Ribeiro, "Connecting the dots: Identifying network structure via graph signal processing," *IEEE Signal Process. Mag.*, vol. 36, no. 3, pp. 16–43, May 2019.
- [25] I. Jablonski, "Graph signal processing in applications to sensor networks, smart grids, and smart cities," *IEEE Sensors J.*, vol. 17, no. 23, pp. 7659–7666, Dec. 2017.
- [26] X. Dong, D. Thanou, P. Frossard, and P. Vanderghenst, "Learning Laplacian matrix in smooth graph signal representations," *IEEE Trans. Signal Process.*, vol. 64, no. 23, pp. 6160–6173, Dec. 2016.
- [27] P. Ferrer-Cid, J. M. Barcelo-Ordinas, and J. Garcia-Vidal, "Graph learning techniques using structured data for IoT air pollution monitoring platforms," *IEEE Internet Things J.*, early access, Mar. 22, 2021, doi: [10.1109/JIOT.2021.3067717](https://doi.org/10.1109/JIOT.2021.3067717).
- [28] V. Kalofolias, X. Bresson, M. Bronstein, and P. Vanderghenst, "Matrix completion on graphs," 2014, *arXiv:1408.1717*. [Online]. Available: <http://arxiv.org/abs/1408.1717>
- [29] F. Monti, M. Bronstein, and X. Bresson, "Geometric matrix completion with recurrent multi-graph neural networks," in *Proc. Adv. Neural Process. Syst.*, 2017, pp. 3697–3707.
- [30] A. Boyarski, S. Vedula, and A. Bronstein, "Deep matrix factorization with spectral geometric regularization," 2020, *arXiv:1911.07255*. [Online]. Available: <https://arxiv.org/abs/1911.07255>
- [31] V. C. Gogineni *et al.*, "Data-driven personalized cervical cancer risk prediction: A graph-perspective," in *Proc. IEEE Int. Workshop Stat. Signal Process.*, 2021, pp. 46–50.
- [32] A. Sandryhaila and J. M. Moura, "Big data analysis with signal processing on graphs," *IEEE Signal Process. Mag.*, vol. 31, no. 5, pp. 80–90, Sep. 2014.
- [33] S. K. Kadambari and S. P. Chepuri, "Product graph learning from multi-domain data with sparsity and rank constraints," 2020, *arXiv:2012.08090*. [Online]. Available: <http://arxiv.org/abs/2012.08090>
- [34] V. Kalofolias and N. Perraudin, "Large scale graph learning from smooth signals," 2017, *arXiv:1710.05654*. [Online]. Available: <https://arxiv.org/abs/1710.05654>
- [35] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.
- [36] K. M. Abadir and J. R. Magnus, *Matrix Algebra*. Cambridge, U.K.: Cambridge Univ. Press, 2005.
- [37] D. Bertsekas, *Nonlinear Programming*. Belmont, MA, USA: Athena Scientific, 1999.
- [38] V. Kalofolias, "How to learn a graph from smooth signals," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2016, pp. 920–929.
- [39] E. Kalnay *et al.*, "The NCEP/NCAR 40-year reanalysis project," *Bull. Amer. Meteorol. Soc.*, vol. 77, pp. 437–471, Mar. 1996.
- [40] Z. Cui, K. Henrickson, R. Ke, and Y. Wang, "Traffic graph convolutional recurrent neural network: A deep learning framework for network-scale traffic learning and forecasting," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 11, pp. 4883–4894, Nov. 2020.
- [41] O. Troyanskaya *et al.*, "Missing value estimation methods for DNA microarrays," *Bioinformatics*, vol. 17, no. 6, pp. 520–525, 2001.
- [42] R. Mazumder, T. Hastie, and R. Tibshirani, "Spectral regularization algorithms for learning large incomplete matrices," *J. Mach. Learn. Res.*, vol. 11, pp. 2287–2322, Jan. 2010.
- [43] S. V. Buuren and K. Groothuis-Oudshoorn, "Mice: Multivariate imputation by chained equations in R," *J. Stat. Softw.*, vol. 45, no. 3, pp. 1–68, 2011.

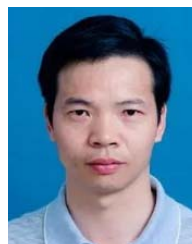


Xiao Jiang received the M.S. degree from Guizhou University, Guizhou, China, in 2020. He is currently pursuing the Ph.D. degree in computer science and technology with the College of Information Science and Engineering, Hunan University, Changsha, China.

His current research interests include machine learning techniques for analyzing network data and its applications in studying disordered systems.



Zean Tian received the Ph.D. degree in materials science and engineering from Hunan University, Changsha, Hunan, China, in 2009. He worked as a Visiting Fellow (February 2010–January 2011) and a Postdoctoral Fellow (January 2010–February 2014) with the University of New South Wales, Australia. He is currently an Associate Professor with the College of Computer Science and Electronic Engineering, Hunan University. He has published over 120 papers in international journals and conference proceedings. His research interests include high-performance/parallel computing and data analysis of disordered systems.



Kenli Li (Senior Member, IEEE) is a Full Professor of Computer Science and Technology with Hunan University. He has published more than 300 papers in international conferences and journals. His main research interests include parallel and distributed processing, supercomputing and cloud computing, high-performance computing for big data, and artificial intelligence. He is an Outstanding Member of CCF. He is currently serves on the Editorial Board of IEEE TRANSACTIONS ON COMPUTERS.